Deep Learning Systems Analysis Report

Plant Leaf Disease Classification Using Convolutional Neural Networks

---

1. Report Overview

This report presents a controlled deep learning experiment comparing two convolutional neural network (CNN) architectures for **plant leaf disease classification** using the PlantVillage dataset. The goal is to evaluate how architectural depth and residual connections affect classification performance across 38 disease and healthy-state categories spanning 14 crop species.

A **Baseline CNN** (shallow, 2-block architecture) is compared against an **Advanced CNN** (deeper residual network with GroupNorm and skip connections) while keeping all training hyperparameters constant. Results show that the Advanced CNN achieves **99.43% test accuracy** compared to the Baseline CNN's **43.12%**, demonstrating the critical importance of architectural design for fine-grained image classification tasks.

---

2. Dataset and Task Description

Task

Multi-class image classification: given an RGB image of a plant leaf, predict the disease category (or healthy status) from 38 possible classes.

Dataset: PlantVillage

The PlantVillage dataset (Hughes & Salathé, 2015) is a publicly available benchmark for automated plant disease detection. It contains **54,305 color images** of plant leaves at a uniform resolution of **256×256 pixels**.

Property	Value
Total images	54,305
Classes	38 (diseases + healthy states)
Crop species	14
Image resolution	256×256 px (RGB)
Source	PlantVillage on Figshare

Data Preparation

The dataset was split using `splitfolders` with a fixed seed (42) for reproducibility:

Split	Samples	Proportion
Train	43,429	80%
Validation	5,417	10%
Test	5,459	10%

Channel-wise normalization statistics were computed from the training set:

Channel	Mean	Std
Red	0.4666	0.1991
Green	0.4891	0.1748
Blue	0.4104	0.2173

Data augmentation was applied only to the training split: - Random horizontal flip (p=0.5) - Random rotation (±10°) - Color jitter (brightness=0.1, contrast=0.1, saturation=0.1, hue=0.02)

A `WeightedRandomSampler` was used during training to mitigate class imbalance by oversampling underrepresented classes (He & Garcia, 2009). This random oversampling strategy assigns higher sampling probabilities to minority classes, ensuring each mini-batch has balanced class representation and preventing the model from being biased toward majority classes.

---

3. Model Architecture and Design Decisions

Baseline CNN

The Baseline CNN is a simple two-block convolutional network inspired by early CNN designs (LeCun et al., 1998):

```
Feature Extractor:
  Conv2d(3→32, 3×3, same) → ReLU → BatchNorm2d → MaxPool2d(2×2)
  Conv2d(32→64, 3×3, same) → ReLU → BatchNorm2d → MaxPool2d(2×2)

Classifier:
  Flatten → Linear(262,144→128) → ReLU → Dropout(0.4)
          → Linear(128→64) → ReLU → Dropout(0.4)
          → Linear(64→38)
```

Design rationale: This architecture serves as a minimal baseline to evaluate what a shallow CNN can achieve on this task. Batch normalization stabilizes training (Ioffe & Szegedy, 2015), and dropout provides regularization (Srivastava et al., 2014). The large flattened feature vector (262,144) connects to the MLP head without spatial pooling, which is computationally expensive but standard for shallow models.

Advanced CNN

The Advanced CNN uses residual blocks with GroupNorm (Wu & He, 2018) and global average pooling (Lin et al., 2014):

```
Stem:   Conv2d(3→32, 3×3) → GroupNorm(8) → ReLU

Stage 1: 2× ResidualGNBlock(32→32, stride=1)     → 256×256
Stage 2: 2× ResidualGNBlock(32→64, stride=2)      → 128×128
Stage 3: 2× ResidualGNBlock(64→128, stride=2)     →  64×64
Stage 4: 2× ResidualGNBlock(128→256, stride=2)    →  32×32
```

`Head: AdaptiveAvgPool2d(1×1) → Flatten → Dropout(0.3) → Linear(256→38)`

Each `ResidualGNBlock` consists of two 3×3 convolutions with GroupNorm and a skip connection (with 1×1 projection when dimensions change), following the residual learning framework (He et al., 2016).

Design rationale: - **Residual connections** enable gradient flow through deep networks, addressing the degradation problem observed in very deep plain networks (He et al., 2016). - **GroupNorm** was chosen over BatchNorm because it is independent of batch size, making it more stable for smaller effective batch sizes on MPS hardware (Wu & He, 2018). - **Global average pooling** replaces the large flattened FC layers, dramatically reducing parameter count and improving spatial invariance (Lin et al., 2014). - **Progressive spatial downsampling** (stride-2 at stages 2–4) builds increasingly abstract feature representations.

---

4. Experimental Comparison

Controlled Variable: Architecture

The experiment isolates the effect of **architecture** by changing only the model structure while keeping all other variables constant:

Parameter	Baseline CNN	Advanced CNN
Architecture	2 conv blocks + MLP	4 residual stages + global pooling
Normalization	BatchNorm	GroupNorm (8 groups)
Skip connections	None	Residual (identity + projection)
Classifier head	Flatten + 3 FC layers	AdaptiveAvgPool + 1 FC layer
Dropout	0.4	0.3
Optimizer	AdamW	AdamW
Learning rate	1e-3	1e-3
Weight decay	1e-4	1e-4
LR Scheduler	CosineAnnealingLR (T_max=30)	CosineAnnealingLR (T_max=30)
Batch size	32	32
Epochs	30	30
Loss function	CrossEntropyLoss	CrossEntropyLoss
Data augmentation	Same	Same
Device	MPS	MPS

The AdamW optimizer (Loshchilov & Hutter, 2019) with CosineAnnealingLR scheduler (Loshchilov & Hutter, 2017) was used for both models, providing stable convergence with a gradual learning rate decay from 1e-3 to 0 over 30 epochs.

---

5. Results and Interpretation

Quantitative Results

Metric	Baseline CNN	Advanced CNN
Best Epoch	27	29
Best Val Accuracy	44.18%	99.43%
Test Accuracy	43.12%	99.43%
Test Loss	1.7000	0.0216
Train Accuracy	44.62%	99.91%
Train Loss	1.6782	0.0040

Head-to-Head Test Comparison

- **Accuracy delta (Advanced CNN — Baseline CNN):** +56.31 percentage points
- **Relative improvement:** +130.59%
- **Loss reduction:** −1.6784
- **Verdict:** The Advanced CNN decisively outperforms the Baseline CNN

Training Dynamics

Baseline CNN: Training loss decreased slowly from 3.58 (epoch 1) to 2.14 (epoch 30), with training accuracy reaching only 33%. The slow convergence and low capacity indicate severe **underfitting**: the Baseline CNN's two convolutional layers cannot extract sufficiently discriminative features for 38 classes.

Advanced CNN: Training loss dropped rapidly from 3.02 (epoch 1) to 0.005 (epoch 30), with accuracy climbing from 13% to 99.87%. The validation curve closely tracked the training curve throughout, reaching 99.43% by epoch 29. The minimal train-validation gap indicates strong generalization without significant overfitting.

The CosineAnnealingLR scheduler played a key role in the Advanced CNN's training — the gradual learning rate decay from 1e-3 to 0 over 30 epochs allowed fine-grained parameter adjustment in later epochs, contributing to the steady improvement from ~97% (epoch 10) to 99.43% (epoch 29).

Concrete Model Behavior: Class-Level Failure Analysis

The per-class accuracy comparison reveals starkly different behavior between the two models.

Baseline CNN — Complete failures (0% accuracy):

Class	Support	Baseline CNN Correct	Baseline CNN Accuracy
Apple____Black_rot	63	0	0.00%
Blueberry____healthy	151	0	0.00%
Cherry_(including_sour)____Powdery_mildew	106	0	0.00%
Pepper,_bell____healthy	149	0	0.00%
Tomato____Septoria_leaf_spot	178	0	0.00%
Tomato____Tomato_mosaic_virus	38	0	0.00%

Baseline CNN — Near-zero accuracy classes:

Class	Support	Baseline CNN Correct	Baseline CNN Accuracy
Cherry_(including_sour)____healthy	86	1	1.16%
Pepper,_bell____Bacterial_spot	101	4	3.96%
Apple____healthy	165	8	4.85%
Soybean____healthy	509	33	6.48%

Baseline CNN — Best-performing class:

Class	Support	Baseline CNN Correct	Baseline CNN Accuracy
Corn_(maize)____healthy	117	117	100.00%

Advanced CNN achieved 92% accuracy across all 38 classes. Every class where the Baseline CNN scored 0% was classified at 100% by the Advanced CNN (except `Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot` at 92.3%, which has subtle grayish lesions that can resemble healthy tissue).

This pattern shows that the Baseline CNN learns only the most visually distinctive class patterns (e.g., corn leaves with their unique shape) and completely fails on classes requiring subtle feature discrimination (e.g., distinguishing between different Tomato diseases that share similar leaf morphology). The Advanced CNN's residual connections and progressive downsampling enable it to build hierarchical feature representations that capture these fine-grained differences.

Visualization Summary

The following plots provide a comprehensive visual comparison between the Baseline CNN and Advanced CNN.

Figure 1: Split Metrics Comparison Loss and accuracy side-by-side for both models across all three data splits (train, validation, test).

Figure 2: Test Head-to-Head Direct test performance comparison with accuracy and loss delta annotations.

Figure 3: Training Curves Epoch-by-epoch loss and accuracy trajectories over 30 training epochs.

Figure 4: Test Confusion Matrices 38×38 confusion matrices showing per-class prediction patterns on the test set.

Figure 5: Per-Class Accuracy Comparison Class-level test accuracy for all 38 categories, highlighting the Baseline CNN's failures and the Advanced CNN's consistency.

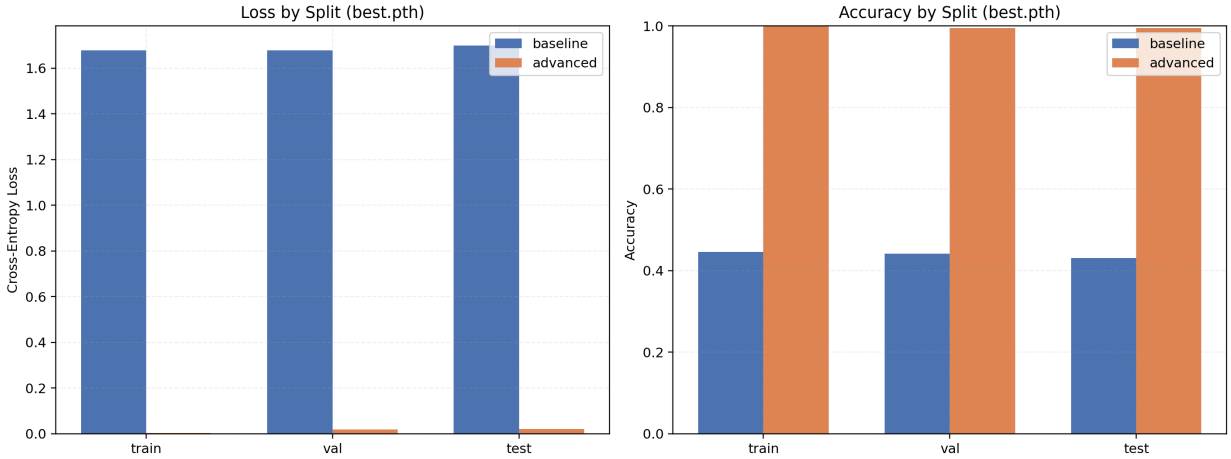

Figure 1: Split Metrics Comparison

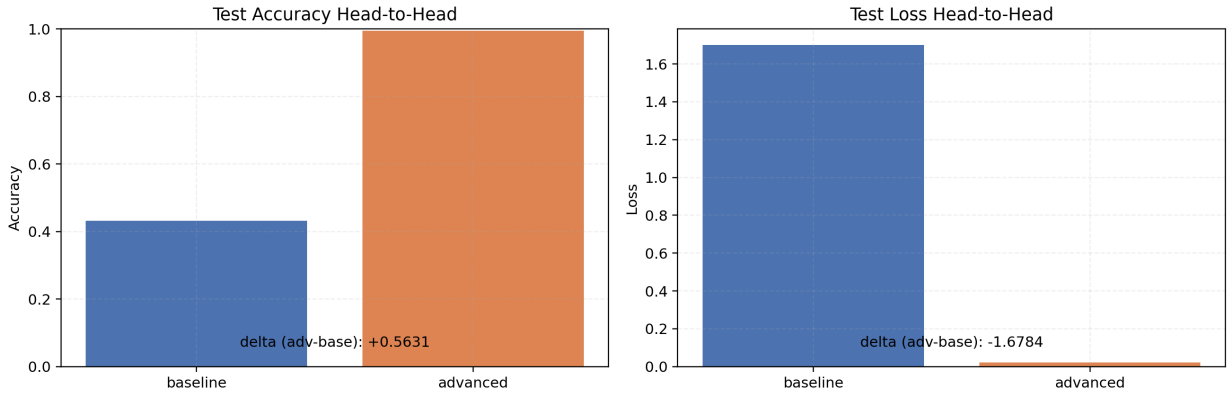

Figure 2: Test Head-to-Head

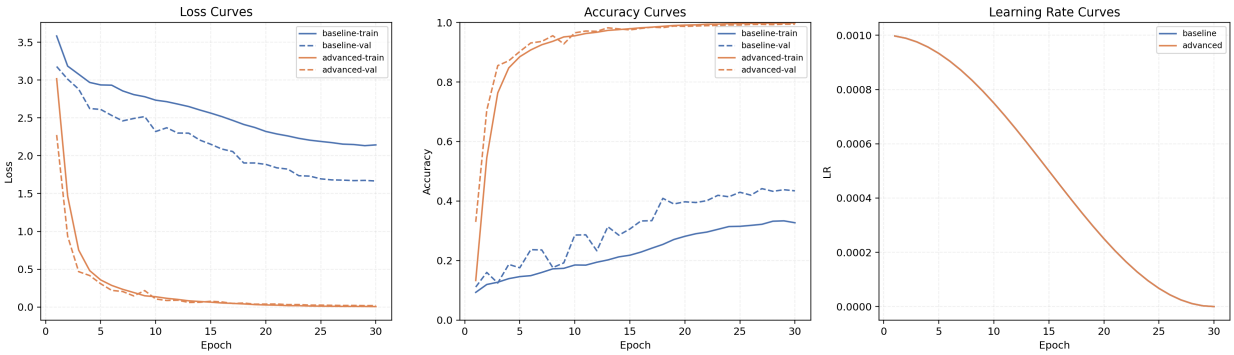

Figure 3: Training Curves Comparison

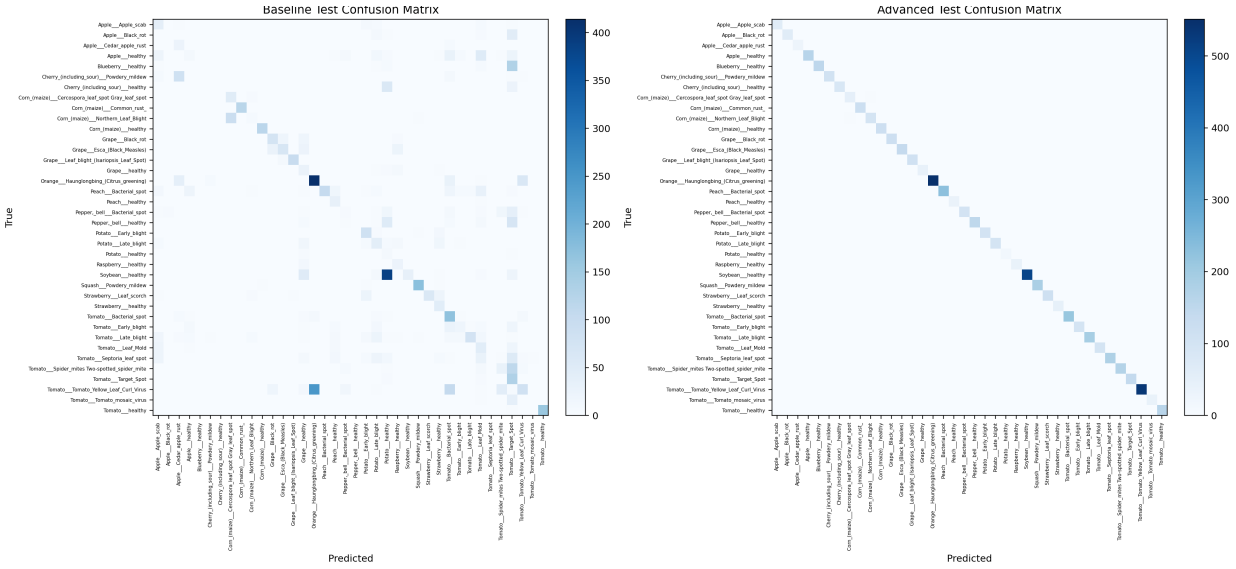

Figure 4: Test Confusion Matrices

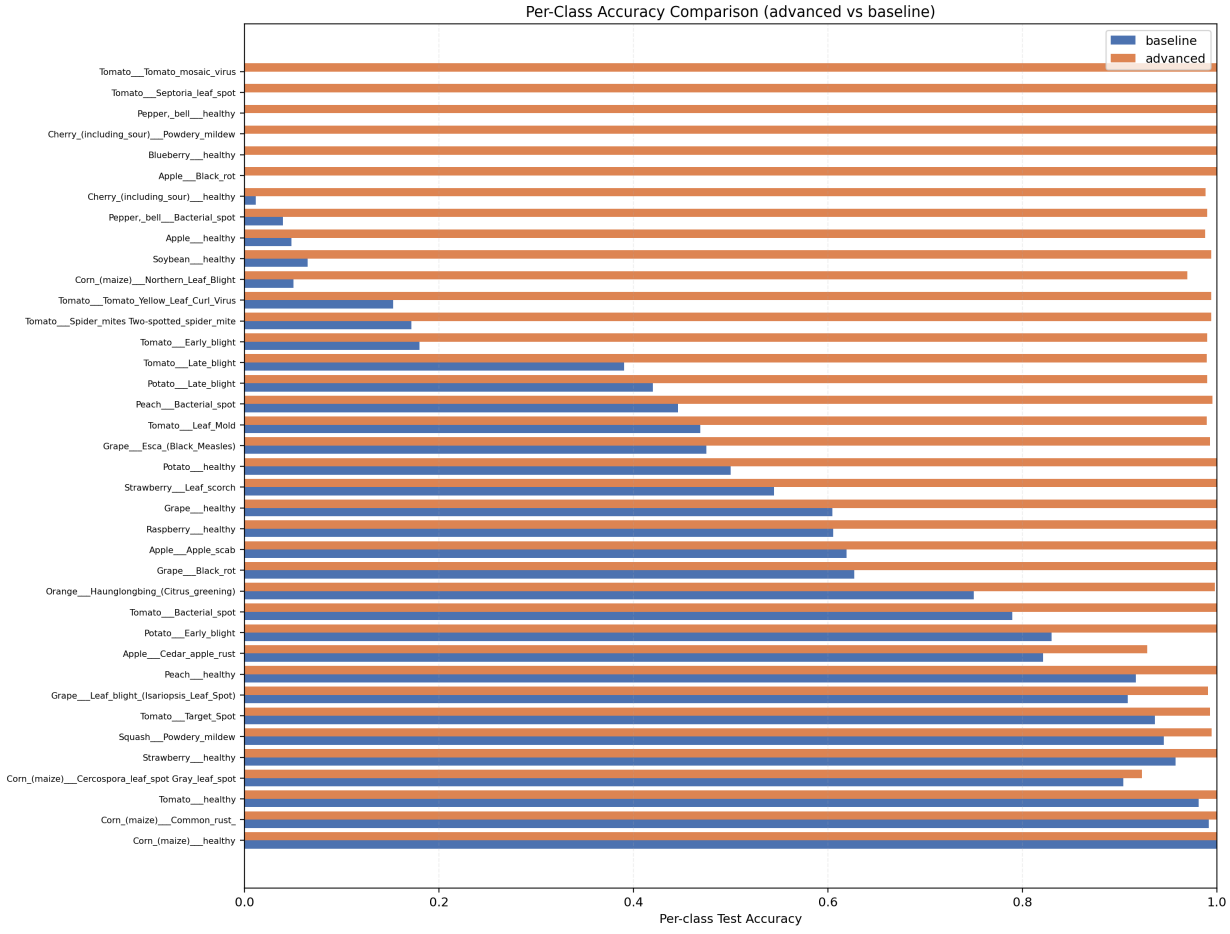

Figure 5: Per-Class Accuracy Comparison

6. Limitations and Risks

Model Limitations

- **Dataset homogeneity:** PlantVillage images are captured in controlled lab conditions with clean backgrounds. Real-world field images include complex backgrounds, varying lighting, multiple diseases per leaf, and occlusion — performance would likely degrade significantly in deployment (Mohanty et al., 2016).
- **Near-perfect accuracy concern:** The Advanced CNN's 99.43% test accuracy on curated lab images may not transfer to field conditions. The model may have learned dataset-specific artifacts (e.g., background patterns, lighting characteristics) rather than true disease features.
- **Limited augmentation:** Only basic augmentation (flip, rotation, color jitter) was applied. More aggressive techniques like CutMix (Yun et al., 2019), MixUp, or random erasing could improve robustness.
- **Single dataset evaluation:** Both models were evaluated only on PlantVillage. Cross-dataset validation (e.g., testing on field-collected images) would better assess generalization.

Data Risks

- **Class imbalance:** Some classes have as few as 16 test samples (`Potato___healthy`) while others have 552 (`Orange___Haunglongbing`). Accuracy metrics on small classes have high variance.
- **No geographic diversity:** PlantVillage images lack geographic variation in plant varieties, soil types, and environmental conditions, limiting global applicability.

---

7. Ethical and Responsible Use

Potential Benefits

- Automated plant disease detection can aid smallholder farmers in developing countries who lack access to plant pathology expertise (Mohanty et al., 2016).
- Early disease detection can reduce crop losses, improve food security, and minimize pesticide overuse.

Ethical Concerns

- **Over-reliance on AI diagnosis:** Farmers may trust model predictions without expert verification, potentially leading to incorrect treatment decisions that cause crop damage or unnecessary chemical use.
- **Access inequality:** Deploying such models requires smartphones and internet connectivity, potentially widening the digital divide between resource-rich and resource-poor farming communities.
- **Bias in dataset composition:** The PlantVillage dataset focuses on 14 crop species prevalent in certain geographic regions. Crops important to other regions (cassava, millet, teff) are underrepresented, creating a bias toward Western agricultural systems.
- **Environmental impact:** Training deep learning models has a carbon cost. The Advanced CNN's training required ~17 hours on MPS (30 epochs × ~33 min/epoch), which is modest but non-trivial.

Responsible Deployment Recommendations

- Models should be deployed as **decision support tools**, not autonomous diagnostic systems.
- Predictions should include confidence scores and "unknown class" detection for inputs outside the training distribution.
- Field validation with local plant pathologists is essential before any agricultural deployment.

---

8. Future Improvements

1. **Transfer learning:** Fine-tuning pretrained models (e.g., ResNet-50, EfficientNet) rather than training from scratch would likely achieve competitive accuracy with significantly less training time and data.
2. **Field image evaluation:** Testing on real-world field datasets (e.g., PlantDoc, iBean) to assess domain shift robustness.
3. **Advanced augmentation:** Techniques like CutMix (Yun et al., 2019), MixUp, and random erasing could improve generalization beyond lab conditions.
4. **Model interpretability:** Applying Grad-CAM or SHAP to visualize which leaf regions the model focuses on, ensuring it learns disease-relevant features rather than background artifacts.
5. **Multi-label classification:** Real-world plants may exhibit multiple diseases simultaneously — extending the model to handle multi-label predictions would increase practical utility.
6. **Model compression:** Deploying on mobile devices (smartphones for field use) requires techniques like knowledge distillation, pruning, or quantization to reduce model size and inference time.

---

9. References

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv preprint arXiv:1511.08060*. https://arxiv.org/abs/1511.08060

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448–456. https://arxiv.org/abs/1502.03167

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. https://doi.org/10.1109/5.726791

Lin, M., Chen, Q., & Yan, S. (2014). Network in network. *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1312.4400

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1608.03983

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *Proceedings of the 7th International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1711.05101

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7, 1419. https://doi.org/10.3389/fpls.2016.01419

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Zemel, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

Wu, Y., & He, K. (2018). Group normalization. *Proceedings of the European Conference on Computer Vision (ECCV)*, 3–19. https://arxiv.org/abs/1803.08494

Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., & Yoo, Y. (2019). CutMix: Regularization strategy to train strong classifiers with localizable features. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 6023–6032. https://arxiv.org/abs/1905.04899